

SmartTaps: Voice-Controlled Interaction for Smartphones (Deliverable 2)

1st Michael Calderin

Department of Engineering Education

University of Florida

Gainesville, FL, USA

michaelcalderin@ufl.edu

Abstract—SmartTaps is an innovative and advanced voice-controlled software framework meticulously designed to enable users to interact with their smartphones in a completely hands-free manner by converting spoken commands into precise and accurate taps on the screen. This system effectively bridges the gap between natural human language and the structured graphical user interfaces (GUIs) commonly found in mobile devices, allowing for seamless navigation and operation without the need for physical touch. By integrating state-of-the-art technologies such as real-time speech recognition, sophisticated vision-language understanding models, and automated device control through the Android Debug Bridge (ADB), SmartTaps represents a significant step forward in enhancing user accessibility and convenience. This preliminary report provides a detailed summary of the first complete implementation of the system as of November 9, 2025, including an in-depth exploration of its architecture, the comprehensive model training process, early evaluation results from testing, and a thoughtful discussion on the various challenges encountered during development, as well as important ethical considerations surrounding the deployment of voice-driven automation technologies. The framework not only facilitates everyday tasks but also holds potential for assisting individuals with disabilities, thereby promoting inclusivity in technology use.

A. PROJECT SUMMARY

The primary purpose of the SmartTaps project is to create a fully hands-free smartphone interface that empowers users to perform taps on their screens entirely through voice commands. This innovative approach allows users to issue natural language directives, such as “tap on the settings icon to adjust brightness” or “tap on the picture of the girl,” which are then intelligently translated into precise tap actions on the device’s screen. By eliminating the need for physical interaction, SmartTaps aims to enhance accessibility for individuals with motor impairments, visual challenges, or those in situations where hands-free operation is essential, such as while driving or multitasking. Furthermore, this project explores the intersection of artificial intelligence and human-computer interaction, pushing the boundaries of how users engage with mobile technology.

Since the submission of Deliverable 1, which focused on initial conceptual designs, substantial progress has been made in bringing the system to a functional state. Key advancements include the development of a specialized dataset consisting of annotated smartphone screenshots, which captures a variety of user interface elements and layouts. It is based

on Google’s screen annotation dataset [1] which provides annotations of UI elements on Android screens. The issue is that those annotations are formatted into complicated strings, so the strings had to be parsed to extract the different UI elements and their associated bounding boxes. The modified dataset was created and visualized using the Jupyter notebook `dataset_creation.ipynb`. Additionally, a multi-modal deep learning model has been implemented and trained via `training.ipynb`, utilizing a fine-tuned Contrastive Language-Image Pretraining (CLIP) architecture to predict bounding boxes that correspond to spoken commands within the visual context of the screen. The integration of these components into a working prototype has been achieved through `app.py`, which now incorporates speech-to-text conversion using OpenAI’s Whisper, visual prediction for element localization, and automated tap execution via the Android Debug Bridge (ADB). This prototype is capable of processing live commands on an Android emulator, demonstrating end-to-end functionality from voice input to device action.

Despite these achievements, the vision-language model is suffering from poor performance and is the main weakness of the project at this time. This is partially due to task complexity in the sense that the dataset has complex screen layouts and the model is supposed to identify one small UI element on the screen. UI elements can also be text or images so the model has to learn a very expansive and diverse set of UI elements. In addition, the model has not been fed the full data available for training because of time and resource constraints. It likely needs a large amount of data due to the complex nature of the task, and it is not seeing enough samples to properly learn the patterns since the full data is not being used. CLIP also expects 224x224 images so mobile screens need to be compressed to maintain their aspect ratio which severely reduces quality and makes UI elements unclear. Improvements for the vision-language model will be the main focus moving forward. Note that originally, Appium, a Python library to control devices, was proposed in Deliverable 1 but this has been replaced by ADB since it makes interacting with Android Studio (the platform that emulates mobile devices) simpler.

B. SYSTEM ARCHITECTURE AND PIPELINE

The SmartTaps system is architected as a modular and sequential pipeline that efficiently transforms user voice in-

puts into actionable device commands, ensuring a smooth flow from perception to execution. An updated architecture diagram, illustrated in Figure 1, depicts the enhanced data flow: starting from voice input, through transcription and preprocessing, model inference, and finally tapping the emulated Android/mobile device.

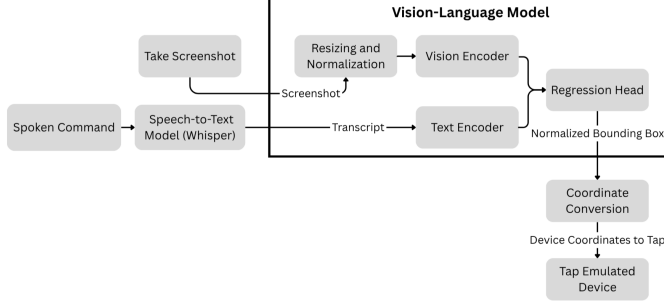


Fig. 1. Updated SmartTaps system architecture and data pipeline, showing the flow from voice input through preprocessing, model inference, and interface execution.

The pipeline commences with the capture of a voice command from the user, which is transcribed into text using OpenAI’s Whisper model. This model is selected for its robustness in handling diverse audio conditions, including accents, background noise, and varying speech rates, providing accurate text representations that form the basis for further processing. From a subjective standpoint, the model seems to be transcribing very well. It has very rarely made mistakes throughout this initial prototyping phase. No plans will be arranged to tweak the model since it does not seem to be the bottleneck for this project. However, a benchmark is still planned in the future to numerically determine its performance.

Next, in the preprocessing stage, a screenshot of the current device state is captured via ADB commands, ADB being a command-line tool that lets a computer interact with emulated Android devices (e.g., taking screenshots and performing taps). The image is then resized to 224x224 pixels to match CLIP’s input requirements, padded to ensure the aspect ratio is preserved. It is then fed to CLIP’s preprocessor which applies additional operations such as normalization to standardize pixel values and enhance feature extraction.

The core model component involves a CLIP-based architecture (Open AI’s clip-vit-base-patch32), comprising a vision encoder (based on a Vision Transformer) that processes the screenshot to extract visual embeddings, a text encoder that generates embeddings from the command text, and a custom regression head that predicts the bounding box coordinates (x1, y1, x2, y2) for the targeted UI element. The target coordinates are normalized in the range [0, 1] and converted to map onto the 224x224 image, taking into account the resizing and padding. For reference, all of the training images are originally 1080x1920 screens before being resized to 224x224.

In terms of the regression head, it is a fully connected neural network that combines the image and text embeddings from CLIP. First, the two embeddings are concatenated and passed

through a linear layer to project them into a 512-dimensional space. This is followed by layer normalization to stabilize training, a ReLU activation for non-linearity, and dropout (20%) for regularization. The output then goes through a second linear layer reducing the dimension to 256, followed again by layer normalization, ReLU activation, and dropout (10%). This module effectively fuses visual and textual features while controlling overfitting and improving the model’s ability to learn complex relationships for bounding box prediction.

Following inference, the interface module scales the normalized bounding box to the device’s resolution and executes a tap command using ADB (e.g., `adb shell input tap x y`). The point to tap is the center of the bounding box. The interface plays a crucial role in taking action based on the AI’s predictions, providing real-time feedback through terminal logs and visual overlays to validate actions and aid in debugging.

This architecture ensures modularity, allowing for independent updates to components like swapping speech models or enhancing vision backends, while maintaining overall system integrity.

C. MODEL IMPLEMENTATION DETAILS

The implementation of the SmartTaps model relies on a suite of powerful frameworks and libraries to handle multimodal data processing and device interaction. Primarily, PyTorch serves as the deep learning backbone, offering flexible tensor operations and GPU acceleration. The CLIP model, pretrained by OpenAI, is utilized for its vision-language capabilities, while OpenAI’s Whisper library manages speech-to-text transcription. Transfer learning is used with the CLIP model, adding a regression head to predict bounding boxes of UI elements. Pandas is used to read and manage CSV annotation data, while Pillow (PIL) handles all image loading, resizing, and padding operations. Matplotlib provides visualization tools to display images, predicted bounding boxes, and training metrics. PyTorch is the framework for building and training the bounding box regression model, including the custom Complete Intersection over Union (CIoU) loss function. Additionally, Transformers from Hugging Face allows integration of the pre-trained CLIP model for extracting image and text embeddings, which are combined for bounding box prediction.

The training setup was conducted on an average gaming PC to facilitate efficient computation. This included an NVIDIA RTX 2060 GPU, AMD Ryzen 7 3700X 8-Core Processor, 16 GB RAM, and at least 10 GB of storage space for project files. This configuration allowed for handling large datasets and complex models without excessive delays. The training regimen spanned 5 epochs, with a batch size of 32 to balance memory usage and gradient stability. A learning rate of 1×10^{-4} was employed to ensure gradual convergence, using the AdamW optimizer for adaptive parameter updates. The model was trained using the CIoU loss, which directly optimizes the predicted bounding boxes by considering overlap, distance between box centers, and aspect ratio consistency.

No data augmentation techniques, such as random cropping, flipping, or color jittering, were applied during training. The model relies solely on the original images and text annotations to learn accurate bounding box predictions.

To ensure reproducibility, the codebase is structured with modular, self-contained scripts under version control. Key features for reproducibility include the following:

- `dataset_creation.ipynb`: Automates the conversion of Google’s screen annotations into a clean CSV file, including visualization for quality checks.
- `training.ipynb`: Includes data loaders, model definition, training loops, validation routines, and metric logging.
- `app.py`: Orchestrates inference by integrating speech processing, model prediction, and ADB execution (screenshot taker and screen tapper).
- Random seeds are fixed across all stochastic operations to ensure reproducibility.
- Detailed documentation of dependencies and hyperparameters is provided to allow exact replication of experiments.

D. INTERFACE PROTOTYPE

The SmartTaps interface prototype is designed to provide a visual and interactive demonstration of the system’s capabilities. Users can provide inputs as spoken commands (e.g., “tap the home button”) via microphone, which are then transcribed. To begin recording a command, the command line is used to listen for a window of time. In future prototypes, this is expected to be replaced with active listening where the system automatically detects when the user is providing a command. Along with the recording, a screenshot is taken with the current state of the screen. The recording and screenshot are fed into respective models, a bounding box is predicted, the center of the bounding box is chosen as the point to tap, and the tap is executed on the emulated device. Ultimately, the user has access to a terminal that they use to record their voice, and they can view taps through the emulated mobile device on Android Studio.

There are a few built-in features for debugging. For example, events are logged in the command line. The system also generates a visualized screenshot with the predicted bounding box overlaid and the point that will be tapped. If coordinates for a tap are wrong and/or fall on a UI element that isn’t clickable, the emulated device won’t show any signs of a tap. Thus, these images are necessary to view predictions and determine the root of errors. Predicted bounding boxes are also shown on both the original screen size and the 224x224 screen to see the screenshot through the model’s lense (and its reduction in quality). These mechanisms were employed to visualize predictions and help with quality assurance on a more general level.

Figure 2 shows a tap prediction when the screen has wild animals and the user says “polar bear.” Notice that the padded image fed into the model is much more blurry, but the model was still able to locate the polar bear. This gives insight into

the backend logic, which is useful for debugging, but hidden to the user.

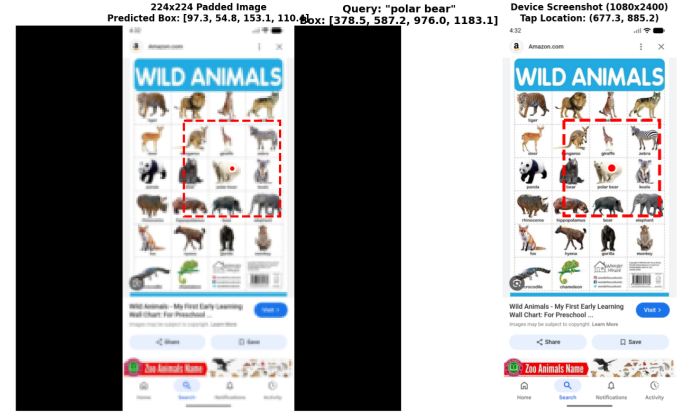


Fig. 2. User said “polar bear” into the microphone and the predicted bounding boxes are shown, typically used for debugging.

The user interface is shown in Figure 3. It requires opening an emulated device through Android Studio and running `app.py`. The device is automatically detected by the Python script. Notice that with the current prototype, the user has to press the “Enter” key to start a 3 second sound recording. That is transcribed, a screenshot is taken, and the model makes its prediction for the tap location. On average, the process from recording to tapping takes 1-2 seconds.

The goal for the next prototype is to remove the need to interact with the command line for sound recordings and simply having the program listen continuously through the microphone, executing commands when sound is detected. This is a limitation for the current prototype. This project is tailored to Android devices so lack of diversity in mobile environments is another limitation. However, annotated data for the iPhone and other mobile devices was not as readily available. Performance on different screen sizes is yet to be tested, but a larger height relative to width will be more blurred when converted to 224x224 which likely degrades prediction quality. Finally, the weak performance of the vision-language model leads to taps in the wrong areas so the model needs to be improved in the next iteration.

E. EARLY EVALUATION AND RESULTS

The early evaluation of SmartTaps focused on quantitative metrics to assess the vision-language model’s ability to accurately localize UI elements based on text commands. Key performance indicators included Intersection-over-Union (IoU), which measures the overlap between predicted and ground-truth bounding boxes, alongside CIoU loss values during training. The first 10,000 samples of data were used and split into 70% for training, 10% for validation, and 20% for testing.

Based on the last epoch that improved the model, training loss was 1.16 and validation loss was 1.21. Training IoU was 0.052 and validation IoU was 0.057. Testing loss was 1.21

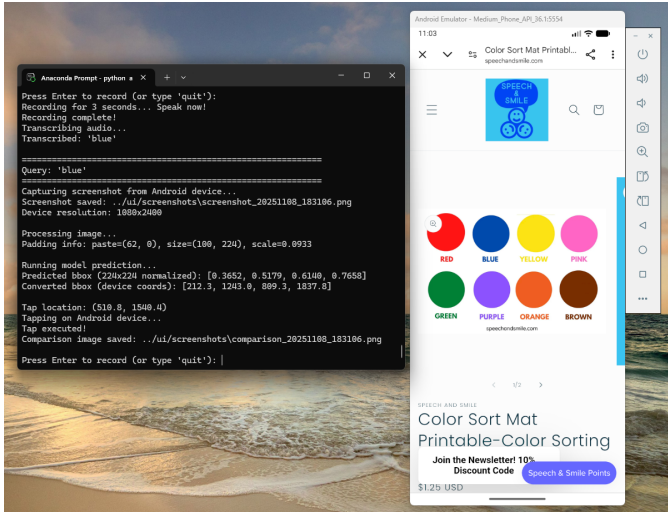


Fig. 3. User interface of SmartTaps with a user speaking “blue” into the microphone.

and testing IoU was 0.058. Note that metrics are means unless otherwise stated.

Figure 4 shows loss and IoU curves. Validation performance seems to be improving across each epoch, but training performance is improving much more rapidly. Overfitting is a high concern, especially since, after inspection, there seem to be some icons that the model has learned much better than others. Since these icons appear in all dataset splits, they could bias results into making it appear validation is improving when it is only improving for a narrow set of UI elements. The last four text and image encoder layers of CLIP were unfrozen since that appeared to lower loss, but more experimentation needs to be done to determine the number of layers to unfreeze. If there are too many unfrozen layers, it could potentially lead to overfitting.

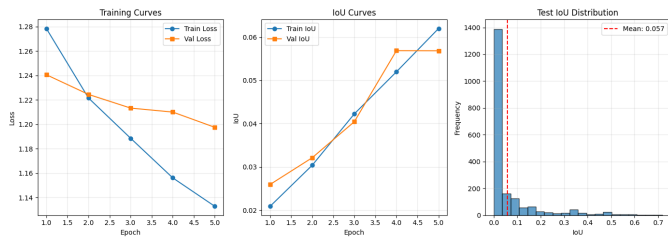


Fig. 4. Training, validation, and testing results based on the last epoch that improved the vision-language model (epoch 4).

The IoU distribution for the test set also hints at UI bias. Very few UI elements are being identified well, while the majority of predictions are completely off (0.00 IoU). Overall, the generalization power of the model is very weak. It needs to be fed more data and/or trained for more epochs, which more data is available, but it will be computationally expensive to train. There also needs to be more experimentation with architecture choices and hyperparameters. Still, there is concern that the dataset is simply too complex and it would take more samples

than available and much greater computing resources to see desired results.

F. CHALLENGES AND NEXT STEPS

During the implementation of SmartTaps, several key challenges arose that affected both model performance and system usability. The primary issue was the suboptimal performance of the vision-language model, with IoU scores around 0.058 and a loss near 1.21 on the test set. This low performance reflects the complexity of detecting small and varied UI elements such as icons, buttons, and text labels in cluttered screen layouts. The dataset, sourced from Google’s screen annotation corpus, contains many annotations, but training was limited to only 10,000 samples (out of roughly 300,000 samples extracted) due to time and computational constraints. As a result, the model lacked sufficient exposure to diverse examples, reducing its ability to generalize effectively.

A major limitation stems from CLIP’s fixed input size of 224x224 pixels, which required compressing high-resolution screenshots from 1080x1920 while maintaining aspect ratios through padding. This resizing severely degraded visual clarity, making it difficult for the model to capture fine details. Signs of overfitting was also observed, with training performance improving faster than validation metrics. There were also biases toward frequently recurring icons. Although unfreezing the last four CLIP layers reduced loss, it risked further overfitting. Resizing images also required transforming/mapping coordinates to new spaces, which was additional effort.

Handling ambiguous commands presented another challenge, as multiple UI elements could match a given description, sometimes resulting in incorrect selections. Compatibility across Android versions and device resolutions remains uncertain, with potential risks from aspect ratio variations. Furthermore, the interface currently depends on manual command-line triggers for voice recording, which undermines its goal of seamless, hands-free operation. The lack of annotated datasets for iOS also restricted cross-platform applicability.

To improve SmartTaps before Deliverable 3, upcoming work will focus on expanding training beyond the initial 10,000 samples and increasing the number of epochs to enhance learning depth. Hyperparameter tuning will target optimal learning rates, batch sizes, and layer freezing strategies, while regularization and data augmentation techniques such as cropping, flipping, and color jittering will be considered. Exploring other architectures or preprocessing methods will help preserve image quality, and latency will be reduced through model quantization and faster device communication methods. Finally, continuous voice recognition will be added to create a more fluid and responsive user experience.

G. RESPONSIBLE AI REFLECTION

The development of SmartTaps has brought to light several ethical, fairness, and privacy issues that must be carefully managed to ensure responsible deployment. Privacy concerns are paramount, as the system processes potentially sensitive

data from screenshots (e.g., personal messages, emails) and voice inputs (e.g., spoken content that could reveal identities or locations). To mitigate this, all processing is currently performed locally, with no data transmission to external servers.

Fairness issues arise from biases in speech recognition, where accents, dialects, or non-native speakers might experience higher error rates in command transcription, potentially excluding certain user groups. Similarly, the vision-language model may underperform on UI themes or languages not well-represented in the dataset, affecting accessibility. To address these, part of the validation process will be understanding how the system's performance is impacted by accents and different screen layouts. This information will be documented, and if time allows, addressed directly by diversifying datasets used.

Reliability and safety are also critical, as incorrect predictions could lead to unintended actions, such as opening wrong apps or altering settings, eroding user trust or causing harm in sensitive contexts. The vision-language model will be refined to lower the chance of incorrect taps. It is the most critical part of this project, requiring special attention and dedicated time.

By proactively tackling these issues through design choices and iterative improvements, SmartTaps aims to uphold ethical standards, promote equitable access, and protect user privacy in its refinement phase.

REFERENCES

- [1] "Google research screen annotation dataset," https://github.com/google-research-datasets/screen_annotation, accessed: 2025-10-16.